

KMP

Knuth-Morris-Pratt

Análise de Complexidade de Tempo

Pedro Garcez · Rafael Ferraz · Denys Leonardo · Felipe Cavalcanti

Teoria da Computação · Prof. Daniel Bezerra

Python + Java

Agenda

- 1 O Problema: Busca de Padrão em String
- 2 Como Funciona o KMP — Tabela LPS e Busca
- 3 Classificação Assintótica: O , Ω , Θ
- 4 Análise de Casos: Melhor, Médio e Pior
- 5 Metodologia Experimental
- 6 Resultados: Python e Java
- 7 Aplicabilidade, Classe P e Reflexão Final

1. O Problema: Busca de Padrão em String

O Problema

Dado um texto T (comprimento n) e um padrão P (comprimento m), encontrar todas as ocorrências de P em T .

Exemplo

$T = \text{"ABABDABACDABABCABAB"}$

$P = \text{"ABABCABAB"}$

✓ Ocorrência encontrada na posição 10

Por que não busca ingênua?

Busca Ingênua

$O(n \cdot m)$

pior caso

KMP

$O(n + m)$

pior caso

Rabin-Karp

$O(n + m)^*$

pior caso

2. Como Funciona: A Tabela LPS

LPS = Longest Proper Prefix which is also Suffix

A tabela LPS pré-processa o padrão e indica quanto podemos avançar quando encontramos uma falha, sem retroceder no ponteiro do texto.

Exemplo: P = "ABABCABAB"

i	0	1	2	3	4	5	6	7	8
P[i]	A	B	A	B	C	A	B	A	B
LPS	0	0	1	2	0	1	2	3	4

Complexidade: **$O(m)$** — Cada caractere do padrão é processado no máximo duas vezes.
A LPS permite deslocar o padrão sem retroceder o ponteiro do texto.

3. Classificação Assintótica

Big-O (O)

$$O(n + m)$$

Limite superior. O KMP nunca realiza mais que $n + m$ comparações — independente do tipo de entrada.

Big-Ω (Omega)

$$\Omega(n + m)$$

Limite inferior. O algoritmo precisa ler ao menos o texto inteiro (n) e construir a LPS completa (m).

Big-Θ (Theta)

$$\Theta(n + m)$$

$O = \Omega$, portanto Θ existe. KMP é assintoticamente ótimo: melhor, médio e pior caso coincidem.

O KMP é um dos poucos algoritmos onde $O = \Omega = \Theta$ — robusto contra qualquer tipo de entrada.

4. Análise de Casos

Melhor Caso

$\approx n$ comparações

Entrada usada:

T = "aaa...a"
P = "baaa...a"

Por quê?

O primeiro caractere do padrão nunca aparece no texto. A LPS nunca é consultada. 1 comparação por posição.

Caso Médio

$O(n + m)$

Entrada usada:

Texto e padrão
aleatórios

Por quê?

Distribuição uniforme sobre alfabeto.
Falhas parciais são raras. Retrocesso via LPS ocorre esporadicamente.

Pior Caso

$\approx 2n$ comparações

Entrada usada:

T = "aaa...a"
P = "aaa...ab"

Por quê?

LPS = [0,1,2,...,m-2,0]. O algoritmo combina m-1 chars, falha no último e retrocede — repetidamente.

5. Metodologia Experimental

Protocolo de Coleta

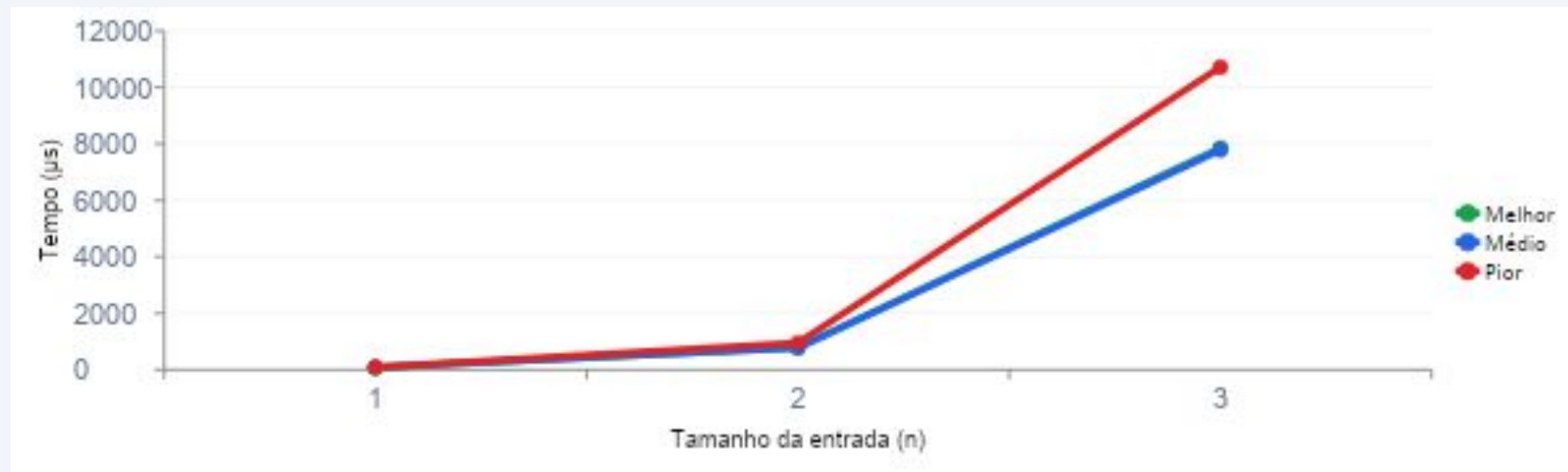
Rodadas cronometradas	30 por cenário
Aquecimento (warmup)	5 rodadas (descartadas)
Tamanhos (n)	1.000 / 10.000 / 100.000
Tamanho do padrão (m)	1% de n
Medição Python	<code>time.perf_counter()</code>
Medição Java	<code>System.nanoTime()</code>
Estatística	Média + desvio-padrão amostral

Ambiente de Execução

Processador	13th Gen Intel(R) Core(TM) i7-13650HX (2.60 GHz)
Memória RAM	16GB DDR5
Sistema Op.	Windows 10/11
Python	3.13.x
Java (JDK)	21

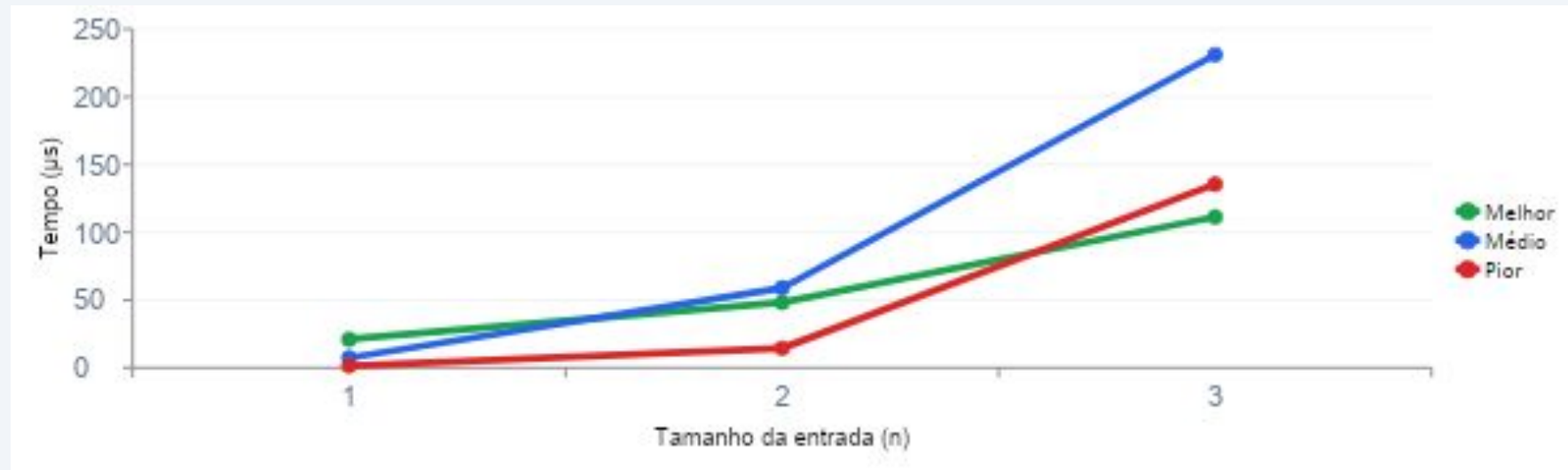
6a. Resultados Experimentais – Python

Caso	n = 1.000	n = 10.000	n = 100.000
Melhor	$77,32 \pm 13,07 \mu s$	$782,32 \pm 79,04 \mu s$	$7.851,24 \pm 284,85 \mu s$
Médio	$77,60 \pm 14,85 \mu s$	$779,99 \pm 48,64 \mu s$	$7.786,46 \pm 316,21 \mu s$
Pior	$90,32 \pm 11,27 \mu s$	$947,90 \pm 162,32 \mu s$	$10.714,53 \pm 345,39 \mu s$

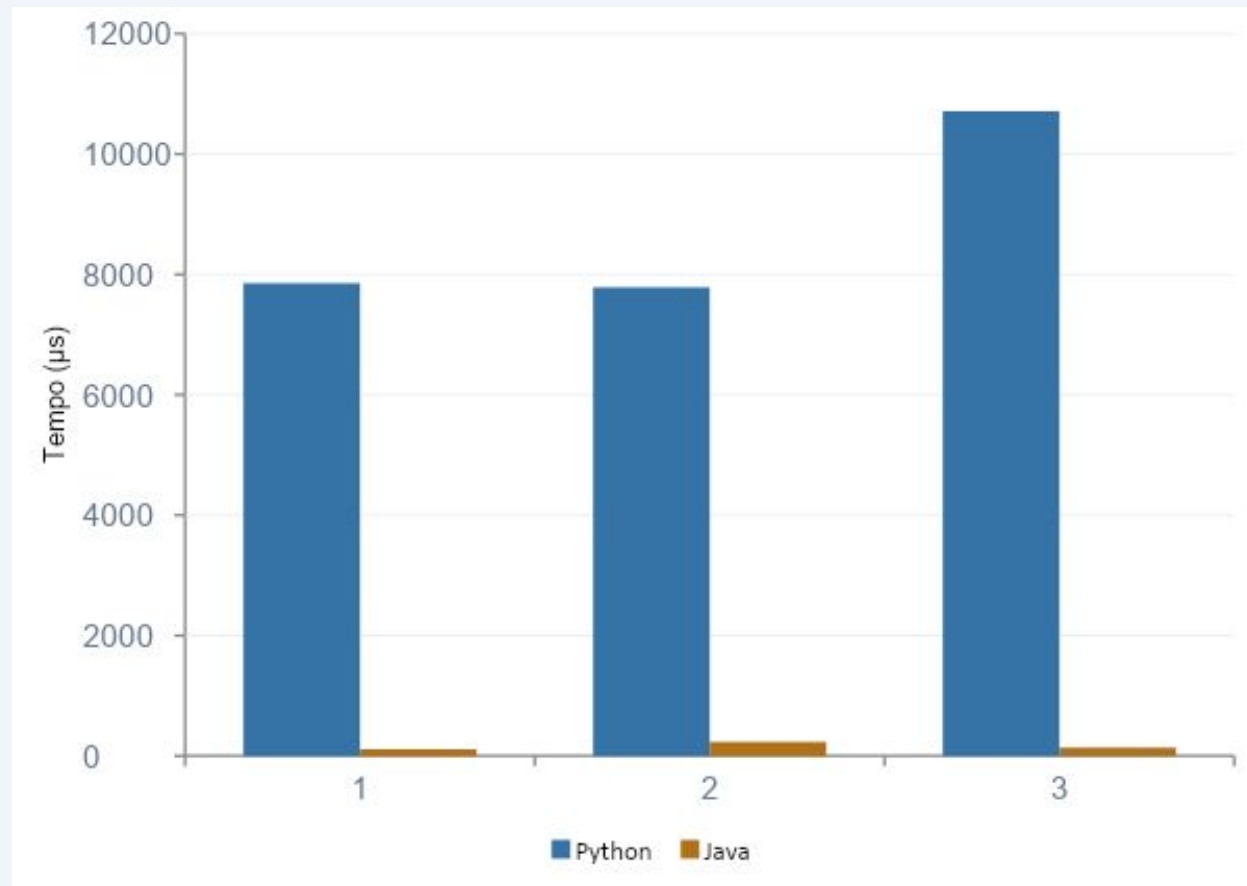


6b. Resultados Experimentais – Java

Caso	n = 1.000	n = 10.000	n = 100.000
Melhor	$20,98 \pm 3,64 \mu\text{s}$	$48,23 \pm 11,68 \mu\text{s}$	$111,33 \pm 42,14 \mu\text{s}$
Médio	$7,41 \pm 6,81 \mu\text{s}$	$58,97 \pm 15,76 \mu\text{s}$	$231,04 \pm 10,98 \mu\text{s}$
Pior	$1,44 \pm 0,06 \mu\text{s}$	$14,40 \pm 1,78 \mu\text{s}$	$135,51 \pm 10,72 \mu\text{s}$



7. Comparação: Python vs. Java (n = 100.000)



Melhor Caso

~70x

Java mais rápido

Caso Médio

~34x

Java mais rápido

Pior Caso

~79x

Java mais rápido

8. Aplicabilidade



Editores de Texto

Busca e substituição em arquivos.
Base do grep e do Ctrl+F.



Bioinformática

Busca de sequências em genomas
com bilhões de pares de base.



Segurança

Deteção de padrões maliciosos em
IDS, antivírus e firewalls.



Redes

Inspeção profunda de pacotes (DPI)
para filtros de conteúdo.



Compressão

Base para LZ77 — usado em ZIP, GZIP
e DEFLATE.



Limitação

Apenas busca exata. Busca
aproximada exige distância de
Levenshtein ou BK-Tree.

9. Reflexão Final: Classe P e NP



KMP pertence à classe P

O KMP resolve a busca exata de padrão em tempo linear $O(n + m)$, que é polinomial em relação ao tamanho da entrada.

Existe um algoritmo determinístico que o resolve eficientemente para qualquer tamanho de entrada.

Complexidade: $\Theta(n + m) \subset \text{DTIME}(n) \subset P$



Problemas NP Relacionados

Substring Comum Mais Longa

$P \text{ — DP } O(n \cdot m)$

Menor Superstring

$NP\text{-completo}$

Busca Aproximada (Edit Dist.)

$P \text{ — DP } O(n \cdot m)$

Expressões Regulares

$P \text{ — Autômatos}$

Conclusões

KMP resolve busca de padrão em $\Theta(n + m)$ — ótimo em todos os casos.

A tabela LPS é a chave: permite deslocar o padrão sem retroceder no texto.

Java foi significativamente mais rápido que Python ($\sim 34\text{--}79\times$) para $n = 100.000$.

Resultados experimentais confirmaram crescimento linear nas duas linguagens.

O algoritmo pertence à classe P e tem ampla aplicabilidade em bioinformática, segurança e compressão.